Lecture 7: Attention

Tokens · queries, keys, values · self- and cross-attention

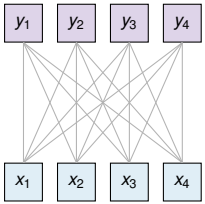
SAiDL

Slides adapted from CS231n (Stanford) Lecture 8 and 3Blue1Brown's "Attention in transformers, visually explained"

Today: Attention

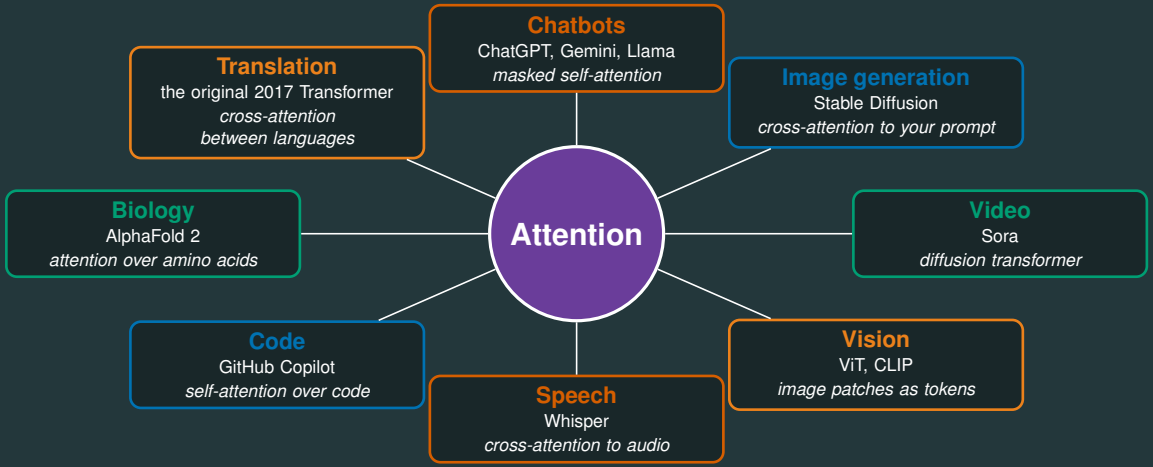
Attention: words **look at each other** and share information

1. Tokens and embeddings
2. **Queries, keys, values**
3. Self-, masked, and cross-attention
4. Why Transformers replaced LSTMs



Not today: the Transformer, multi-head attention

Attention is everywhere



Different data, same idea: turn it into **tokens**, let the tokens **attend** to each other.

Text to vectors: tokenization

1. Tokenize

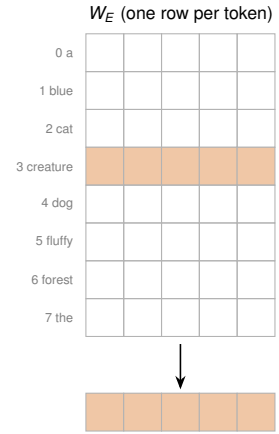
`"a fluffy blue creature"` → a | fluffy | blue | creature

2. Look up IDs

→ `[0, 5, 1, 3]`

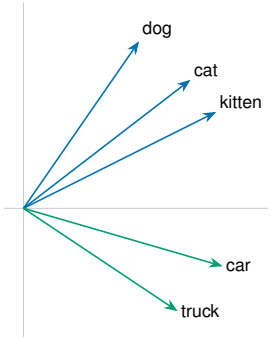
3. Embed: take that row of the table W_E

W_E is learned, so similar words get similar rows.

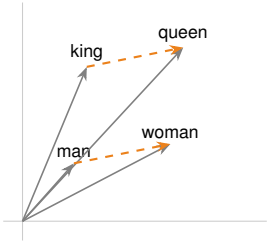


CS231n L8: slide 97

Embeddings are arrows



Similar words point the **same way**



Directions carry meaning

Illustrative 2D pictures; real embeddings have hundreds of numbers.

The problem: meaning depends on context

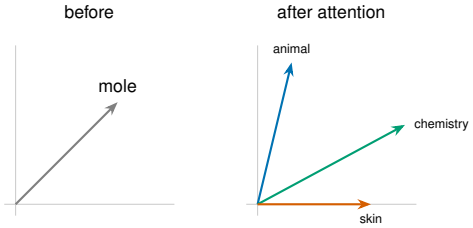
“American shrew **mole**”

“One **mole** of carbon dioxide”

“Take a biopsy of the **mole**”

The lookup gives “mole” the **same vector** every time.

Attention: nudge each word’s vector using the words around it.



Queries, Keys, Values: three roles

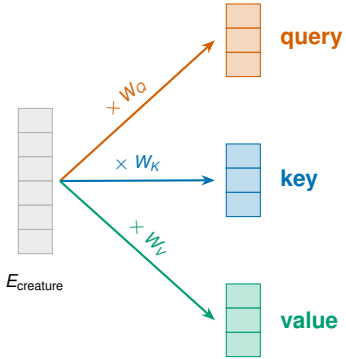
Query: what am I looking for?

Key: what can I be found by?

Value: what do I hand over?

$$q = W_Q E \quad k = W_K E \quad v = W_V E$$

Same three learned matrices for every word.



The Query

$$q = W_Q E$$

A question the word asks about its context.

Toy query “channels”:
1: looking for an **adjective**
2: looking for a **noun**

word	query	asks
fluffy	[0, 1]	“where’s my noun?”
blue	[0, 1.2]	“where’s my noun?”
creature	[1.5, 0]	“any adjectives for me?”

Example: “creature”

$$\underbrace{\begin{bmatrix} 1.5 & 0 \\ 0 & 0 \end{bmatrix}}_{W_Q} \underbrace{\begin{bmatrix} 1 \\ 0.5 \end{bmatrix}}_{E_{\text{creature}}} = \begin{bmatrix} 1.5 \\ 0 \end{bmatrix}$$

Real models learn their own channels; “adjective” and “noun” are just our story.

The Key

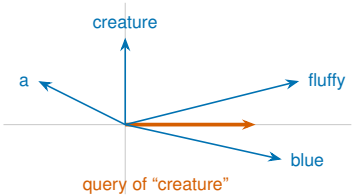
$$k = W_K E$$

A label that lets queries find the word.

Toy key channels:

1: I **am** an adjective
2: I **am** a noun

word	key	says
a	$[-1,\ 0.5]$	“not an adjective”
fluffy	$[2,\ 0.5]$	“adjective!”
blue	$[1.8,\ -0.4]$	“adjective”
creature	$[0,\ 1]$	“noun”



Queries and keys live in the **same** space, so we can compare them.

Matching: the dot product

$$q \cdot k = (\text{ask}_1 \times \text{offer}_1) + (\text{ask}_2 \times \text{offer}_2)$$



- Orange: creature’s query
- Pointing the same way, with a longer arrow ⇒ bigger score

Worked example: scores

Creature's query $q = [1.5,\ 0]$ against every key:

word	key	query · key	score
a	$[-1,\ 0.5]$	$1.5 \times (-1) + 0 \times 0.5$	-1.5
fluffy	$[2,\ 0.5]$	$1.5 \times 2 + 0 \times 0.5$	**3.0**
blue	$[1.8,\ -0.4]$	$1.5 \times 1.8 + 0 \times (-0.4)$	**2.7**
creature	$[0,\ 1]$	$1.5 \times 0 + 0 \times 1$	0.0

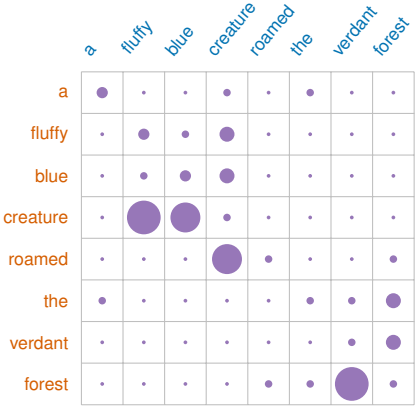
CS231n L8: slides 37–38

Every query vs. every key: the grid

Row: word asking (query)
Column: word answering (key)

Big dots:

- creature → fluffy, blue
- roamed → creature
- forest → verdant



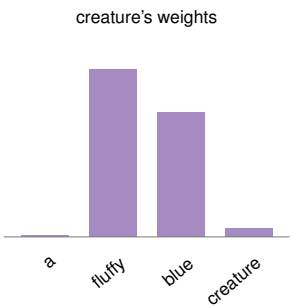
dot size = query · key (illustrative)

Scores $\rightarrow$ weights: softmax

$w_j = e^{s_j} / \sum_l e^{s_l}$: all positive, sum to 1

word	score	e^{score}	weight
a	-1.5	0.22	0.006
fluffy	3.0	20.09	**0.555**
blue	2.7	14.88	**0.411**
creature	0.0	1.00	0.028
total		36.19	1

Bigger gaps between scores $\Rightarrow$ sharper focus.



The Value

$$v = W_V E$$

What a word hands over.

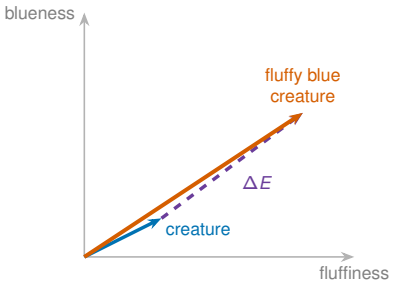
word	value	gives
a	[0, 0, 0]	nothing
fluffy	[1, 0, 0]	fluffiness
blue	[0, 1, 0]	blueness
creature	[0, 0, 1]	creature-ness

Blend with the weights:

$\Delta E = 0.006\, v_{\text{a}} + 0.555\, v_{\text{fluffy}} + 0.411\, v_{\text{blue}} + 0.028\, v_{\text{creature}}$

$= [0.555,\ 0.411,\ 0.028]$

$E' = E + \Delta E$: **nudge**, don't replace.

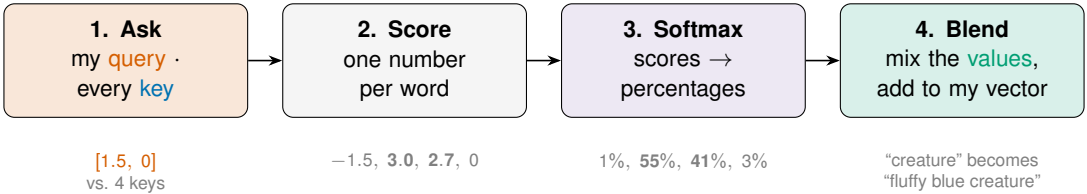


Why three separate vectors?

word	query (asks)	key (is)	value (gives)
fluffy	“my noun?”	adjective	fluffiness
blue	“my noun?”	adjective	blueness
creature	“my adjectives?”	noun	creature-ness

- What I look for $\neq$ what I am $\neq$ what I give
- fluffy and blue: **same key**, **different values**
- W_Q, W_K, W_V are learned; “adjectives” is just our story

Attention in four steps (for one word)



Every word does this **at the same time**.

Full example: every word at once

"fluffy blue creature", all three words together:

word	query	key	value
fluffy	$[0,\ 1]$	$[2,\ 0.5]$	$[1, 0, 0]$
blue	$[0,\ 1.2]$	$[1.8,\ -0.4]$	$[0, 1, 0]$
creature	$[1.5,\ 0]$	$[0,\ 1]$	$[0, 0, 1]$

Full example: every word at once

"fluffy blue creature", all three words together:

word	query	key	value
fluffy	$[0,\ 1]$	$[2,\ 0.5]$	$[1, 0, 0]$
blue	$[0,\ 1.2]$	$[1.8,\ -0.4]$	$[0, 1, 0]$
creature	$[1.5,\ 0]$	$[0,\ 1]$	$[0, 0, 1]$

Scores (row asks, column answers)

	fluffy	blue	creature
fluffy	0.5	-0.4	**1.0**
blue	0.6	-0.48	**1.2**
creature	**3.0**	**2.7**	0

Full example: every word at once

"fluffy blue creature", all three words together:

word	query	key	value
fluffy	$[0,\ 1]$	$[2,\ 0.5]$	$[1, 0, 0]$
blue	$[0,\ 1.2]$	$[1.8,\ -0.4]$	$[0, 1, 0]$
creature	$[1.5,\ 0]$	$[0,\ 1]$	$[0, 0, 1]$

Scores (row asks, column answers)

	fluffy	blue	creature
fluffy	0.5	-0.4	**1.0**
blue	0.6	-0.48	**1.2**
creature	**3.0**	**2.7**	0

Weights (softmax per row)

	fluffy	blue	creature
fluffy	0.327	0.133	**0.540**
blue	0.316	0.107	**0.576**
creature	**0.558**	**0.414**	0.028

Full example: every word at once

"fluffy blue creature", all three words together:

word	query	key	value
fluffy	$[0,\ 1]$	$[2,\ 0.5]$	$[1, 0, 0]$
blue	$[0,\ 1.2]$	$[1.8,\ -0.4]$	$[0, 1, 0]$
creature	$[1.5,\ 0]$	$[0,\ 1]$	$[0, 0, 1]$

Scores (row asks, column answers)

	fluffy	blue	creature
fluffy	0.5	-0.4	**1.0**
blue	0.6	-0.48	**1.2**
creature	**3.0**	**2.7**	0

Weights (softmax per row)

	fluffy	blue	creature
fluffy	0.327	0.133	**0.540**
blue	0.316	0.107	**0.576**
creature	**0.558**	**0.414**	0.028

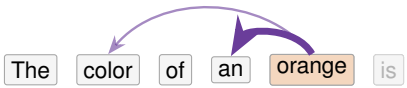
Outputs

fluffy $\rightarrow [0.33,\ 0.13,\ 0.54]$: "I'm describing a creature"

creature $\rightarrow [0.56,\ 0.41,\ 0.03]$: "I'm fluffy and blue"

Example: “The color of an orange is ___”

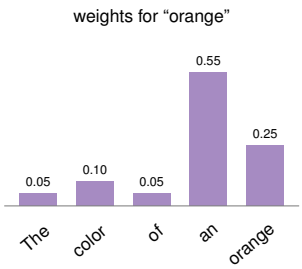
Step 1: “orange” works out **which** orange it is (it can only look back)



Query of “orange”: *“am I a fruit or a colour here?”*

word	key (says)	value (gives)
an	“a **thing** comes next”	“you’re a noun”
color	“I’m a property”	“colour is the topic”
The, of	filler	little

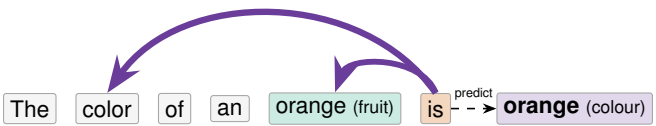
⇒ “orange” becomes **orange (the fruit)**



illustrative numbers

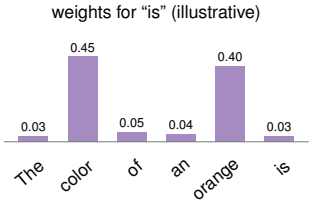
Example: “The color of an orange is ___”

Step 2: “is” must predict the next word, so it asks what the question is about



Query of “is”: *“what is being asked, and about what?”*

word	key (says)	value (gives)
color	“I’m the property asked for”	“answer is a colour”
orange	“I’m the object”	“it’s the fruit”
others	filler	little

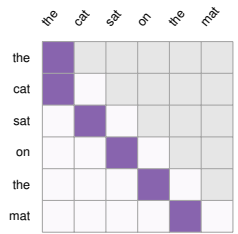


$\Rightarrow$ “is” carries **colour of the fruit orange** $\rightarrow$ “orange”
Same token, two meanings: input “orange” $=$ **fruit**, predicted “orange” $=$ **colour**. Context decides.

What do real attention patterns look like?

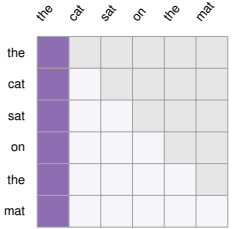
Patterns found in trained models (sketches):

Look at the previous word



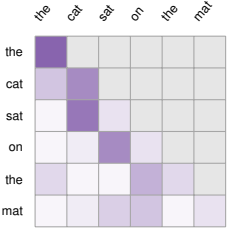
word order, short phrases

Look at the first word



a “resting spot” (weights must sum to 1)

Look at related words



“sat” $\rightarrow$ who sat? “cat”. “mat” $\rightarrow$ “sat on”

Learned, not programmed. Grey $=$ masked future words.

e.g. Olsson et al. 2022 (previous-token heads); Xiao et al. 2023 (attention sinks)

Attention in code: one word at a time

```
import numpy as np

def softmax(scores):
    e = np.exp(scores)
    return e / e.sum()

def attend(query, keys, values):
    scores = np.array([query @ k for k in keys]) # 1-2
    weights = softmax(scores) # 3
    return sum(w * v for w, v in zip(weights, values)) # 4

# a fluffy blue creature
keys = np.array([[-1, .5], [2, .5], [1.8, -.4], [0, 1]])
values = np.array([[0,0,0], [1,0,0], [0,1,0], [0,0,1]])

attend(np.array([1.5, 0]), keys, values)
# -> [0.555, 0.411, 0.028]
```

Lines 1–2: query · every key
Line 3: softmax
Line 4: weighted sum of values

Once per word ⇒ self-attention.

The whole thing in one line

How papers write it:

$$\text{Attention}(Q, K, V) = \underbrace{\text{softmax}}_{\text{step 3}}\left(\frac{\overbrace{QK^\top}^{\text{steps 1–2: all the dot products}}}{\sqrt{d}}\right)\underbrace{V}_{\text{step 4}}$$

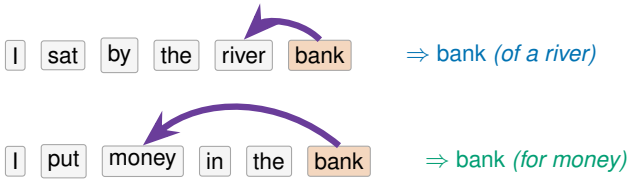
Q, K, V: all queries, keys, values stacked, one row per word.

($\sqrt{d}$ just keeps the numbers from getting too big.)

CS231n L8: slides 38, 45

Self-Attention

The words of **one** sentence look at each other.



- Same word, same starting vector; different neighbours ⇒ different meaning
- Attention ignores word order by itself, so a **position vector** is added to each embedding

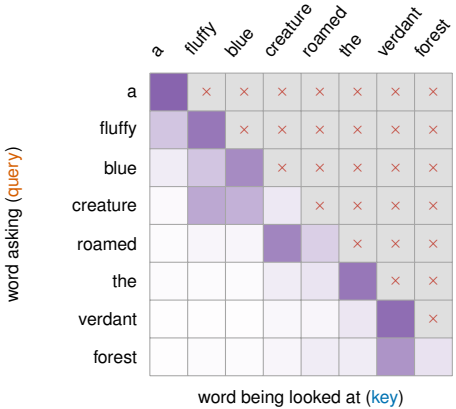
CS231n L8: slides 47–60

Masked Self-Attention

For next-word prediction, a word may not look **ahead**.

Set future scores to $-\infty$:
$e^{-\infty} = 0 \Rightarrow$ weight 0

Used in GPT-style models.



CS231n L8: slides 61–62

Cross-Attention

Translating French → English:

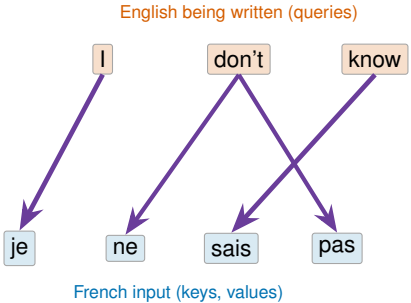
"je ne sais pas" → "I don't know"

Queries: from the English being written

Keys and **values**: from the French sentence

Each English word asks: *"which French words am I translating?"*

Same steps as before; only **where the vectors come from** changes.



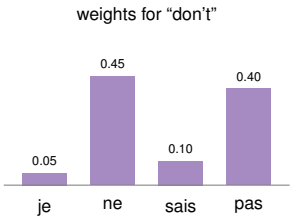
Cross-attention, step by step: writing “don’t”

Query of “don’t”: *“where is the negation?”*

French	key (says)	value (gives)
je	“I’m the subject”	“I”
ne	“negation, part 1”	“not”
sais	“I’m the verb: know”	“know”
pas	“negation, part 2”	“not”

“don’t” collects “not” from **two** French words, with “sais” sitting between them.

Next, writing “know” puts almost all its weight on “sais”.

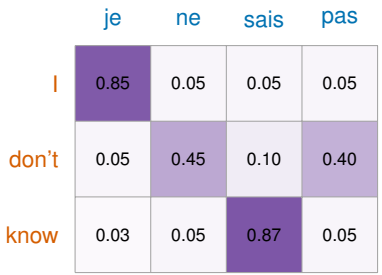


illustrative numbers

Cross-attention: the word-match grid

Row: English word being written (query)
Column: French word (key)

- Not one-to-one: 4 French words $\rightarrow$ 3 English words
- "don't" $\leftarrow$ "ne" $+$ "pas", which aren't even next to each other
- Nobody tells the model which words match; it **learns** this from many translated sentences



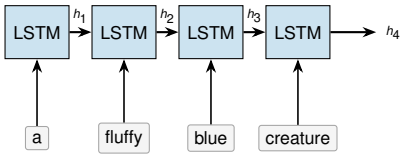
each row sums to 1 (illustrative)

Before 2018: LSTMs

Read text **one word at a time**, carrying **one memory vector** h_t:

$$h_t = f(h_{t-1},\ x_t)$$

Used in Google Translate (2016) and ELMo (2018).



Hochreiter & Schmidhuber 1997; Wu et al. 2016; Peters et al. 2018

Why Transformers won

1. Parallel. LSTM: N steps in a row. Attention: all words at once.

Much faster to train on GPUs.

2. No bottleneck. LSTM squeezes everything into h_t.

LSTMs struggled to use words from far back.

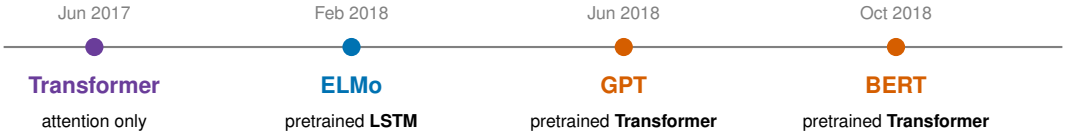
Catch: every word looks at every word, which gets expensive for long texts.

	LSTM	**Attention**
steps in a row	N	1
word-to-word path	N	1

Vaswani et al. 2017; Khandelwal et al. 2018

CS231n L8: slides 80–84

2018: the switch



Pretraining on huge text rewards whatever trains fastest ⇒ Transformers.
BERT set records on 11 tasks; LSTMs faded from NLP.

Peters et al. 2018; Radford et al. 2018; Devlin et al. 2018

Summary

- **Tokens** $\rightarrow$ embedding vectors; context is missing
- **Attention**: query $\cdot$ key $\rightarrow$ scores $\rightarrow$ softmax $\rightarrow$ blend values $\rightarrow E' = E + \Delta E$
- **Self**: a sentence looks at itself (mask the future for next-word prediction)
- **Cross**: one sequence looks at another
- **Beat LSTMs**: parallel, no memory bottleneck

Next time: the Transformer: attention (gather information) + MLP (process it), stacked many times

Reference: the equations (self-attention)

Inputs: N tokens $t_1, \ldots, t_N$, $X \in \mathbb{R}^{N \times d_{\text{model}}}$ with rows $x_i = W_E[t_i] + p_i$ (embedding + position)

Projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \qquad W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

Scores: $S = \dfrac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{N \times N}, \qquad S_{ij} = \dfrac{q_i \cdot k_j}{\sqrt{d_k}}$

Causal mask (for next-word prediction): $M_{ij} = 0$ if $j \le i$, $M_{ij} = -\infty$ if $j > i$

Weights (softmax over each row): $A_{ij} = \dfrac{\exp(S_{ij} + M_{ij})}{\sum_l \exp(S_{il} + M_{il})}$

Outputs: $Y = AV, \qquad y_i = \sum_j A_{ij}\, v_j$

Update: $x_i' = x_i + y_i W_O$ ($W_O \in \mathbb{R}^{d_v \times d_{\text{model}}}$ maps back to embedding size)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

CS231n L8: slides 45, 61

Reference: cross-attention, multi-head, cost

Cross-attention (queries from $X_A \in \mathbb{R}^{N_A \times d}$, keys/values from $X_B \in \mathbb{R}^{N_B \times d}$):

$$Q = X_A W_Q, \quad K = X_B W_K, \quad V = X_B W_V, \qquad Y = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \in \mathbb{R}^{N_A \times d_v}$$

Multi-head attention (not covered today; H heads, each with its own $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$):

$$\mathrm{head}_h = \mathrm{Attention}\big(XW_Q^{(h)},\ XW_K^{(h)},\ XW_V^{(h)}\big), \qquad \mathrm{MultiHead}(X) = [\mathrm{head}_1; \ldots; \mathrm{head}_H]\, W_O$$

usually $d_k = d_v = d_{\text{model}}/H$. GPT-3: $d_{\text{model}} = 12288$, $H = 96$, $d_k = 128$.

Cost per layer: attention $O(N^2 d)$ compute and an $N \times N$ weight matrix; an RNN/LSTM is $O(Nd^2)$ but needs N sequential steps.

Notation note: the slides wrote $q = W_Q E$ with column vectors. Papers stack words as rows and write $q_i = x_i W_Q$; it's the same thing transposed.

Vaswani et al. 2017; Brown et al. 2020 (GPT-3)

References

- 3Blue1Brown, *Attention in transformers, visually explained* (Deep Learning, Chapter 6), the “fluffy blue creature” and “mole” examples come from here
- CS231n (Stanford), Lecture 8: Attention and Transformers
- Vaswani et al. *Attention Is All You Need*. NeurIPS 2017
- Jay Alammar, *The Illustrated Transformer* (blog)
- Hochreiter & Schmidhuber. *Long Short-Term Memory*. Neural Computation 1997
- Khandelwal et al. *Sharp Nearby, Fuzzy Far Away: How Neural Language Models Use Context*. ACL 2018
- Peters et al. (ELMo) 2018; Radford et al. (GPT) 2018; Devlin et al. (BERT) 2018